

## **2. Formulário Técnico**

**Pontifícia Universidade Católica de Minas Gerais**

Instituto de Ciências Exatas e Informática

AED III – Algoritmos e Estruturas de Dados III

### **Trabalho Prático – Fase III**

Relacionamento N:N com Tabela Intermediária

Sistema de Gerenciamento de Clínica Médica

Prof. Walisson Ferreira de Carvalho

Belo Horizonte – 2026

## 2. Formulário Técnico

### 1. Visão Geral da Fase III

Esta fase dá continuidade ao sistema de Gerenciamento de Clínica Médica desenvolvido nas Fases I e II. O objetivo principal é a implementação de um relacionamento N:N entre as entidades Médico e Especialidade, realizado por meio de uma tabela intermediária persistida em arquivo binário, com indexação própria.

Além do relacionamento N:N, a fase exige: (a) pelo menos uma funcionalidade de listagem ordenada utilizando travessia da Árvore B+; (b) compatibilidade total com os arquivos binários já existentes das fases anteriores; e (c) interface gráfica sem uso de console.

#### 1.1 Entidades Relacionadas

|                                       |                                                                                                        |
|---------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Entidade A</b>                     | Médico (medicos.dat)                                                                                   |
| <b>Entidade B</b>                     | Especialidade (especialidades.dat)                                                                     |
| <b>Tabela Intermediária</b>           | MedicoEspecialidade (medico_especialidade.dat)                                                         |
| <b>Cardinalidade</b>                  | N:N – um médico pode ter várias especialidades; uma especialidade pode ser exercida por vários médicos |
| <b>Índice da Tabela Intermediária</b> | Hash Extensível (medico_esp_hash.hdir / .hbkt)                                                         |

## 2. Formulário Técnico

### 2.1 Qual foi o relacionamento N:N escolhido e quais tabelas ele conecta?

O relacionamento N:N escolhido foi entre Médico e Especialidade. A justificativa é direta: na prática clínica, um mesmo médico pode ser especialista em mais de uma área (ex.: Cardiologia e Clínica Geral), e uma mesma especialidade agrupa vários médicos. Esse é um relacionamento típico do domínio médico e satisfaz plenamente o requisito da Fase III.

A tabela intermediária MedicoEspecialidade conecta as entidades principais e persiste o vínculo no arquivo `medico_especialidade.dat`, seguindo o mesmo padrão de cabeçalho e exclusão lógica (lápide) adotado em todas as demais tabelas do sistema.

### 2.2 Qual estrutura de índice foi utilizada? Justifique a escolha.

Para a tabela intermediária MedicoEspecialidade, foi utilizado o Hash Extensível. A justificativa é técnica e alinhada com o padrão de acesso previsto para a tabela associativa:

- A operação dominante sobre a tabela intermediária é a busca direta por ID do registro (chave primária inteira), para a qual o Hash Extensível oferece acesso em  $O(1)$  amortizado, superior ao  $O(\log n)$  da Árvore B+.
- A tabela intermediária não possui necessidade de listagem ordenada por ela própria — as listagens ordenadas são exigidas nas entidades principais (Médico, Paciente, Consulta), onde a Árvore B+ já está corretamente empregada.
- O requisito de consulta ordenada da Fase III é atendido pela Árvore B+ já existente nos DAOs de Médico, Paciente e Consulta, por meio do endpoint `GET /api/ordenar`.

A Árvore B+ está presente em MedicoDAO (`medicos_btree.btree`), PacienteDAO (`pacientes_btree.btree`) e ConsultaDAO (`consultas_btree_data.btree`), demonstrando sua aplicação prática para recuperação de dados em ordem sem uso de ordenação em memória principal.

### 2.3 Como foi implementada a chave composta da tabela intermediária?

A tabela MedicoEspecialidade possui chave primária simples (`int id`) gerada automaticamente pelo mecanismo de BaseDAO (campo `ultimoid` no cabeçalho). A chave composta lógica (`idMedico`, `idEspecialidade`) é verificada por unicidade na camada de aplicação: o método `criar()` de MedicoEspecialidadeDAO percorre os registros ativos antes da inserção e rejeita duplicatas com base no par (`idMedico`, `idEspecialidade`).

Essa abordagem preserva a compatibilidade com o BaseDAO genérico e garante a integridade do relacionamento N:N. Cada registro armazena quatro campos de tamanho fixo:

| Campo | Tipo / Tamanho |
|-------|----------------|
|-------|----------------|

## 2. Formulário Técnico

|                    |                  |
|--------------------|------------------|
| id                 | int (4 bytes)    |
| ativo (lápide)     | boolean (1 byte) |
| idMedico           | int (4 bytes)    |
| idEspecialidade    | int (4 bytes)    |
| Total por registro | 13 bytes         |

### 2.4 Como é feita a busca eficiente de registros por meio do índice?

O HashExtensivel mapeia id → offset em disco. Na busca por ID direto, o DAO calcula o bucket via hash(id) & mascara, lê o bucket do arquivo .hbkt e compara as entradas para obter o offset, que é usado para posicionar o RandomAccessFile e ler os 13 bytes do registro.

Para as consultas relacionais (todas as especialidades de um médico ou todos os médicos de uma especialidade), a implementação atual percorre todos os slots ativos do arquivo medico\_especialidade.dat filtrando por idMedico ou idEspecialidade. Essa varredura sequencial é eficiente porque o arquivo intermediário tende a ser pequeno em comparação com os arquivos de entidades principais.

```
// Busca: especialidades de um médico
public List<Integer> idEspecialidadesPorMedico(int idMedico) throws IOException {
    List<Integer> res = new ArrayList<>();
    for (MedicoEspecialidade me : listarTodos()) {
        if (me.getIdMedico() == idMedico) res.add(me.getIdEspecialidade());
    }
    return res;
}
```

### 2.5 Como o sistema trata a integridade referencial?

A integridade referencial é tratada na camada de serviço (ClinicaHttpServer) da seguinte forma:

- Inserção de vínculo: antes de criar, o método criar() de MedicoEspecialidadeDAO verifica se o par (idMedico, idEspecialidade) já existe entre os registros ativos, evitando duplicatas.
- Exclusão de Médico: ao deletar um médico, o serviço remove logicamente todos os vínculos associados ao idMedico na tabela intermediária (exclusão em cascata via lápide).
- Exclusão de Especialidade: ao deletar uma especialidade, o serviço remove logicamente todos os vínculos associados ao idEspecialidade.
- Consulta cruzada: os métodos idEspecialidadesPorMedico e idMedicosPorEspecialidade retornam apenas registros com ativo=true, ignorando lápides.

### 2.6 Como foi organizada a persistência da nova tabela?

## 2. Formulário Técnico

O arquivo `medico_especialidade.dat` segue rigorosamente o mesmo padrão de todas as demais tabelas do sistema:

|                                  |                                                                                             |
|----------------------------------|---------------------------------------------------------------------------------------------|
| <b>Cabeçalho (12 bytes)</b>      | <code>int numRegistros   int ultimold   int tamanhoReg</code>                               |
| <b>Exclusão lógica (lápide)</b>  | campo boolean ativo no byte 4 de cada registro                                              |
| <b>Tamanho fixo por registro</b> | 13 bytes (sem necessidade de realocação)                                                    |
| <b>Arquivo de índice</b>         | <code>medico_esp_hash.hdir</code> (diretório) + <code>medico_esp_hash.hbkt</code> (buckets) |
| <b>Localização</b>               | diretório <code>data/</code> junto aos demais arquivos binários                             |

O padrão de cabeçalho com 3 inteiros (`numRegistros`, `ultimold`, `tamanhoReg`) e o uso de boolean como lápide são herdados de `BaseDAO<T>`, garantindo consistência entre todas as tabelas do sistema.

### 2.7 Integração com o CRUD das tabelas principais

A tabela intermediária `MedicoEspecialidade` se integra ao restante do sistema pelos seguintes pontos de integração:

- Endpoint REST GET `/api/vinculos?medico={id}`: retorna a lista completa de objetos Especialidade associados ao médico, obtida por busca hash em `EspecialidadeDAO` para cada ID retornado por `idEspecialidadesPorMedico`.
- Endpoint REST GET `/api/vinculos?especialidade={id}`: retorna a lista completa de objetos Medico que possuem a especialidade, usando `idMedicosPorEspecialidade`.
- Endpoint REST POST `/api/vinculos`: cria um novo vínculo a partir de `{idMedico, idEspecialidade}` enviados no corpo JSON.
- Endpoint REST DELETE `/api/vinculos`: remove logicamente o vínculo (lápide) a partir do par `{idMedico, idEspecialidade}`.
- Exclusão em cascata: a exclusão de um Médico ou Especialidade dispara `removerVinculo()` para todos os registros relacionados na tabela intermediária.

### 2.8 Estrutura de diretórios e módulos após a Fase III

```
clinica/
├── src/clinica/
│   ├── Main.java                # Ponto de entrada
│   ├── controller/
│   │   └── ClinicaHttpServer.java # Servidor HTTP + rotas REST
│   ├── dao/
│   │   └── BaseDAO.java          # Persistência genérica (cabeçalho +
lápide)
│   │   ├── MedicoDAO.java        # Hash + B+ (índice por ID e ordenação)
│   │   ├── EspecialidadeDAO.java # Hash Extensível
│   │   ├── MedicoEspecialidadeDAO.java # NOVO - tabela N:N com Hash
│   │   └── ConsultaDAO.java      # Hash + B+ (data e valor)
```

## 2. Formulário Técnico

```

├── PacienteDAO.java          # Hash + B+
├── UsuarioDAO.java          # Hash (autenticação)
├── index/
│   ├── ArvoreBMais.java     # B+ persistida em arquivo binário
│   ├── HashExtensivel.java  # Hash Extensível (.hdir + .hbkt)
│   └── OrdenacaoExterna.java # Ordenação por intercalação
├── model/
│   ├── Medico.java
│   ├── Especialidade.java
│   ├── MedicoEspecialidade.java # NOVO - modelo da tabela intermediária
│   ├── Consulta.java
│   └── Paciente.java (via UsuarioDAO)
├── data/                    # Arquivos binários gerados em runtime
│   ├── medico_especialidade.dat # NOVO
│   ├── medico_esp_hash.hdir     # NOVO
│   ├── medico_esp_hash.hbkt     # NOVO
│   ├── medicos.dat / medicos_hash.* / medicos_btree.*
│   ├── especialidades.dat / especialidades_hash.*
│   ├── consultas.dat / consultas_hash.* / consultas_btree.*
│   ├── pacientes.dat / pacientes_hash.* / pacientes_btree.*
│   └── usuarios.dat / usuarios_hash.*
└── run.sh                   # Script de compilação e execução

```

## 2. Formulário Técnico

## 3. Funcionalidades Implementadas na Fase III

### 3.1 CRUD da Tabela Intermediária

Todas as operações CRUD estão disponíveis via API REST no endpoint `/api/vinculos`:

|                                                             |                                                                                |
|-------------------------------------------------------------|--------------------------------------------------------------------------------|
| <b>POST</b> <code>/api/vinculos</code>                      | Cria vínculo entre Médico e Especialidade (verifica duplicata automaticamente) |
| <b>GET</b><br><code>/api/vinculos?medico={id}</code>        | Lista todas as Especialidades de um Médico (navegação N:N)                     |
| <b>GET</b><br><code>/api/vinculos?especialidade={id}</code> | Lista todos os Médicos de uma Especialidade (navegação N:N inversa)            |
| <b>DELETE</b> <code>/api/vinculos</code>                    | Remove logicamente o vínculo (exclusão com lápide)                             |

### 3.2 Listagem Ordenada via Árvore B+

A Fase III exige pelo menos uma funcionalidade de listagem ordenada usando Árvore B+, sem ordenação em memória principal. Isso é atendido por múltiplas funcionalidades já presentes e expandidas:

|                                                                            |                                                                           |
|----------------------------------------------------------------------------|---------------------------------------------------------------------------|
| <b>GET</b><br><code>/api/ordenar?entidade=medicos&amp;campo=nome</code>    | Listagem de médicos em ordem crescente de ID via travessia in-order da B+ |
| <b>GET</b><br><code>/api/ordenar?entidade=pacientes&amp;campo=nome</code>  | Listagem de pacientes em ordem crescente via B+                           |
| <b>GET</b><br><code>/api/ordenar?entidade=consultas&amp;campo=data</code>  | Listagem de consultas ordenadas por data via B+                           |
| <b>GET</b><br><code>/api/ordenar?entidade=consultas&amp;campo=valor</code> | Listagem de consultas ordenadas por valor via B+                          |
| <b>Parâmetro</b> <code>&amp;ordem=desc</code>                              | Inverte a ordem do resultado retornado pela B+                            |

O método `listarOrdenado()` de `ArvoreBMais` realiza a travessia das folhas encadeadas da árvore B+ (cada folha possui ponteiro next para a próxima), retornando pares [chave, offset] em ordem crescente de chave. O DAO então lê cada registro diretamente do arquivo .dat pelo offset, sem qualquer ordenação adicional em memória.

### 3.3 Navegação Bidirecional N:N

## 2. Formulário Técnico

O relacionamento é navegável em ambas as direções, conforme exigido pelo requisito 2 da Fase III:

- Dado um Médico → listar todas as suas Especialidades: GET /api/vinculos?medico={id}
- Dada uma Especialidade → listar todos os Médicos que a possuem: GET /api/vinculos?especialidade={id}

Em ambos os casos, o sistema retorna objetos completos (não apenas IDs), enriquecendo a resposta com os dados do arquivo .dat correspondente via busca por hash.



## 2. Formulário Técnico

## 4. Estrutura dos Arquivos Binários

### 4.1 Formato do Cabeçalho (padrão BaseDAO)

Todos os arquivos .dat do sistema — incluindo o novo `medico_especialidade.dat` — possuem cabeçalho uniforme de 12 bytes:

|                            |                                                       |
|----------------------------|-------------------------------------------------------|
| <b>Bytes 0–3</b>           | int numRegistros (registros com ativo=true)           |
| <b>Bytes 4–7</b>           | int ultimold (último ID gerado, para auto-incremento) |
| <b>Bytes 8–11</b>          | int tamanhoReg (tamanho em bytes de cada slot)        |
| <b>A partir do byte 12</b> | Slots de tamanho fixo (tamanhoReg bytes cada)         |

### 4.2 Formato do Registro MedicoEspecialidade

|                   |                                                                |
|-------------------|----------------------------------------------------------------|
| <b>Bytes 0–3</b>  | int id (chave primária)                                        |
| <b>Byte 4</b>     | boolean ativo (lápide: true=ativo, false=excluído logicamente) |
| <b>Bytes 5–8</b>  | int idMedico (FK para medicos.dat)                             |
| <b>Bytes 9–12</b> | int idEspecialidade (FK para especialidades.dat)               |
| <b>Total</b>      | 13 bytes por registro (tamanho fixo)                           |

### 4.3 Arquivos de Índice Hash Extensível

O Hash Extensível é persistido em dois arquivos complementares:

- `medico_esp_hash.hdir`: diretório com profundidade global e vetor de offsets para os buckets.
- `medico_esp_hash.hbkt`: buckets com pares (chave, offset), profundidade local e ponteiro para overflow.

O mesmo par de arquivos é utilizado por todos os demais DAOs do sistema (`medicos_hash.*`, `especialidades_hash.*`, etc.), garantindo consistência arquitetural.

## 2. Formulário Técnico

### 5. Atendimento aos Critérios de Avaliação

| Critério                           | Descrição                                                                         | Pontos |
|------------------------------------|-----------------------------------------------------------------------------------|--------|
| Modelagem e implementação          | Implementação correta do relacionamento N:N com CRUD funcional                    | 1,0    |
| Uso e justificativa do índice      | Escolha adequada da estrutura (B+ ou Hash) e coerência técnica da justificativa   | 1,5    |
| Integração com as fases anteriores | Compatibilidade com os arquivos e estrutura da Fase II                            | 0,5    |
| Documentação técnica               | Clareza e completude das respostas do formulário                                  | 1,0    |
| Vídeo                              | Vídeo demonstrando o funcionamento do trabalho, focando nas novas funcionalidades | 1,0    |

#### 5.1 Verificação dos Requisitos Específicos da Fase III

|                                              |                                                                                                                    |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>Req. 1 – Chave primária composta</b>      | Atendido: par (idMedico, idEspecialidade) verificado por unicidade no criar(); chave técnica id gerada por BaseDAO |
| <b>Req. 2 – Navegação bidirecional</b>       | Atendido: GET /api/vinculos?medico e GET /api/vinculos?especialidade                                               |
| <b>Req. 3 – Justificativa do índice</b>      | Atendido: seção 2.2 justifica o Hash Extensível para a tabela intermediária                                        |
| <b>Req. 4 – B+ em listagem ordenada</b>      | Atendido: /api/ordenar usa travessia de folhas da B+ em MedicoDAO, PacienteDAO e ConsultaDAO                       |
| <b>Req. 5 – Padrão de cabeçalho e lápide</b> | Atendido: BaseDAO uniformiza cabeçalho de 12 bytes e lápide boolean em todos os arquivos                           |
| <b>Req. 6 – Sem console como interface</b>   | Atendido: interface via HTTP server com frontend web (sem System.out/Scanner como UI)                              |
| <b>Req. 7 – Formulário técnico</b>           | Atendido: seção 2 deste documento responde às 8 perguntas do formulário                                            |